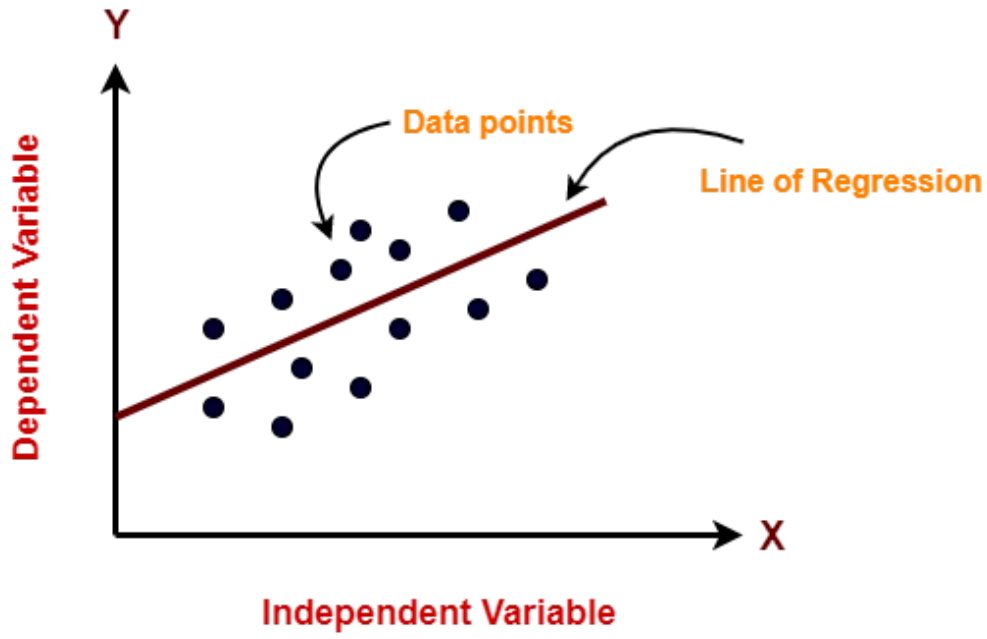
Linear Regression:

Linear Regression is a supervised learning algorithm in machine learning, which is widely used for solving regression problems. Regression is a type of machine learning problem where the goal is to predict a continuous output variable based on one or more input variables.

In Linear Regression, the goal is to find the best-fitting linear equation to describe the relationship between the input variables (also known as predictors or features) and the output variable (also known as the response variable).



The equation for a simple linear regression model can be written as follows:

y = b0 + b1 * x

Here, y is the dependent variable (the variable we are trying to predict), x is the independent variable (the predictor or feature), b0 is the intercept term (the value of y when x is zero), and b1 is the slope coefficient (the change in y for a unit change in x).

The goal of Linear Regression is to find the best values for b0 and b1 such that the line best fits the data points, minimizing the errors or the difference between the predicted values and the actual values.

Types of Linear Regression?

Simple Linear Regression: In simple linear regression, there is only one independent variable (also known as the predictor or feature) and one dependent variable (also known as the response variable). The goal of simple linear regression is to find the best-fitting line to describe the relationship between the independent and dependent variable. The equation for a simple linear regression model can be written as:

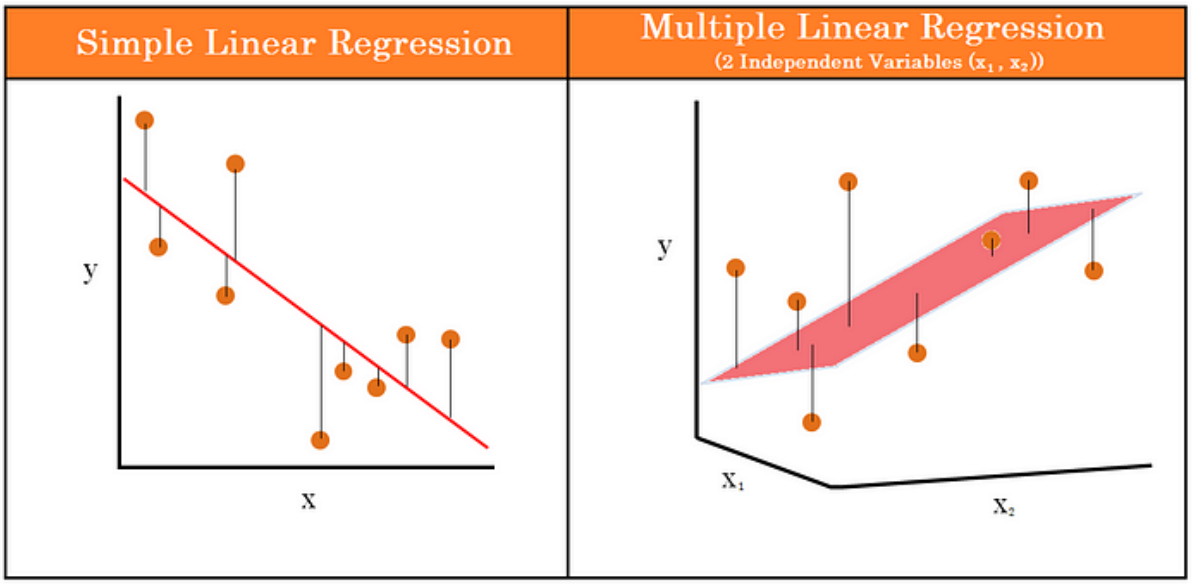
Y = b0 + b1 * X

Here, Y is the dependent variable, X is the independent variable, b0 is the intercept term, and b1 is the slope coefficient.

Multiple Linear Regression: In multiple linear regression, there are multiple independent variables and one dependent variable. The goal of multiple linear regression is to find the best-fitting line to describe the relationship between the independent variables and the dependent variable. The equation for a multiple linear regression model can be written as:

Y = b0 + b1 * X1 + b2 * X2 + … + bn * Xn

Here, Y is the dependent variable, X1, X2, …, Xn are the independent variables, b0 is the intercept term, and b1, b2, …, bn are the slope coefficients.



To find the best-fitting line in a linear regression model, we use a process called "**ordinary least squares (OLS) regression**". This process involves calculating the sum of the squared differences between the predicted values and the actual values for each data point, and then finding the line that minimizes this sum of squared errors.

For simple Linear Regression

$$m = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

OLS Formula (Multiple LR)

Let:

- **X** = input matrix (n × p), where
  - *n* = number of observations
  - *p* = number of features (with a column of 1s for the intercept)
- **y** = target vector (n × 1)
- **β̂** = estimated coefficients (p × 1)

OLS Coefficient Estimate:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Cost function

The cost function, also known as the loss function or objective function, is a measure of how well the model is performing. In linear regression, the cost function is used to calculate the difference between the predicted values and the actual values, also known as the residuals or errors.

Mean Squared Error (MSE), which is calculated as the average of the squared differences between the predicted and actual values:

$MSE = (1/n) * \Sigma(y_i — \hat{y}_i)^2$

Here, n is the number of data points, y_i is the actual value, and $\hat{y}_i$ is the predicted value.

Gradient Descent

Gradient Descent is an optimization algorithm used to minimize the cost function in linear regression models. The goal of gradient descent is to find the values of the intercept and slope coefficients, b0 and b1, that minimize the cost function by iteratively adjusting the values of these coefficients.

The basic idea behind gradient descent is to calculate the gradient of the cost function, which is the direction of steepest descent, and update the values of the coefficients in the opposite direction of the gradient. The learning rate, also known as the step size, determines how large of a step is taken in each iteration.

Gradient Descent

$$y = mx + b$$

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)^2$$

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

$$D_m = \frac{1}{n}\sum_{i=0}^{n}2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n}x_i(y_i - \bar{y}_i)$$

$$D_c = \frac{-2}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)$$

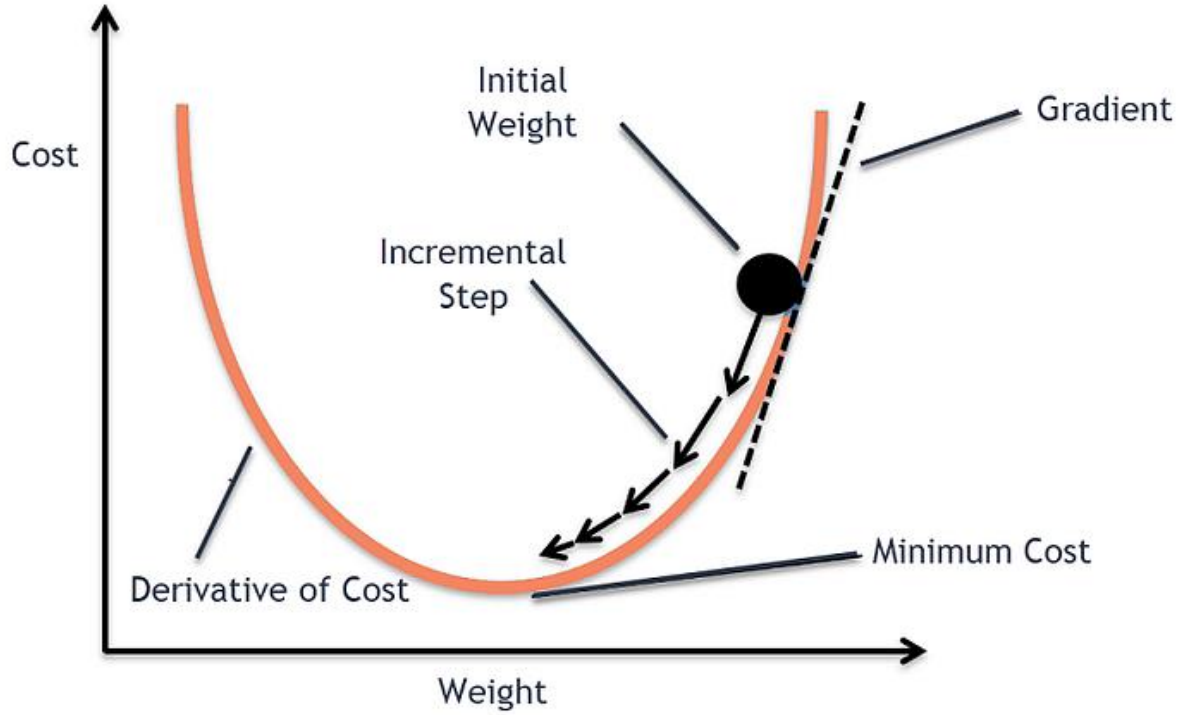
$$m = m - L \times D_m$$

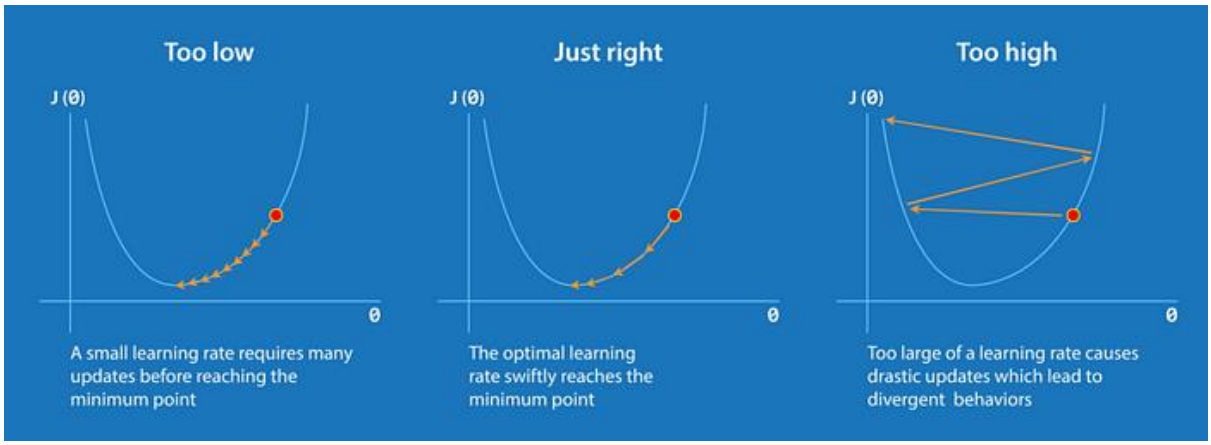
$$c = c - L \times D_c$$

The algorithm works as follows:

1. Initialize the values of b0 and b1 to random values.
2. Calculate the predicted values for the given input data using the current values of b0 and b1.
3. Calculate the cost function using the predicted values and the actual values.
4. Calculate the gradient of the cost function with respect to b0 and b1.
5. Update the values of b0 and b1 by taking a step in the opposite direction of the gradient, with a step size determined by the learning rate.
6. Repeat steps 2–5 until the cost function is minimized or a maximum number of iterations is reached.

The choice of the learning rate is critical in gradient descent, as a learning rate that is too large can cause the algorithm to overshoot the minimum and a learning rate that is too small can cause the algorithm to converge slowly.





There are different variations of gradient descent, including batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, each with their own advantages and disadvantages. The choice of gradient descent algorithm depends on the specific problem and the requirements of the model.

Different types of Gradient descents are

- **Batch Gradient Descent**
- **Stochastic Gradient Descent**
- **Mini batch Gradient Descent**

Batch Gradient Descent

In batch gradient we **use the entire dataset to compute the gradient of the cost function for each iteration of the gradient descent and then update the weights.**

Since we use the entire dataset to compute the gradient convergence is slow.

If the dataset is huge and contains millions or billions of data points then it is memory as well as computationally intensive.

Update w_i

Temp.	Visibility	Actual o/p	Predicted o/p
73	10	1	0
50	2	1	1
32	1	0	1
80	5	1	1
23	.1	0	1

Batch Gradient descent

Batch gradient descent uses the entire dataset to calculate each iteration of gradient descent

Advantages of Batch Gradient Descent

- Theoretical analysis of weights and convergence rates are easy to understand

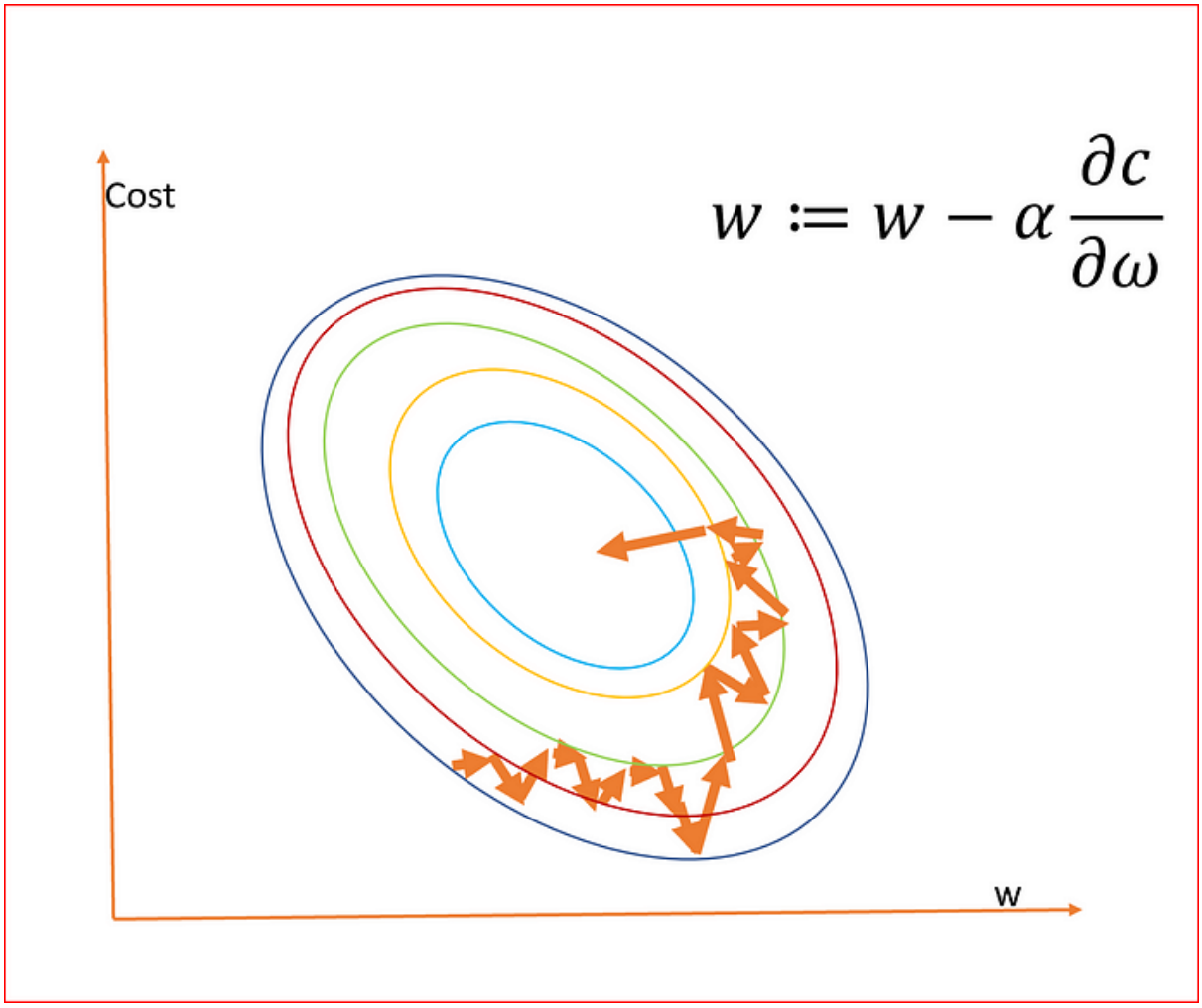
Disadvantages of Batch Gradient Descent

- Perform redundant computation for the same training example for large datasets
- Can be very slow and intractable as large datasets may not fit in the memory
- As we take the entire dataset for computation we can update the weights of the model for the new data

Stochastic Gradient descent

In stochastic gradient descent we **use a single datapoint or example to calculate the gradient and update the weights with every iteration**.

we first need to **shuffle the dataset so that we get a completely randomized dataset**. As the dataset is randomized and weights are updated for each single example, update of the weights and the cost function will be noisy jumping all over the place as shown below



Random sample helps to arrive at a global minima and avoids getting stuck at a local minima.

Learning is much faster and convergence is quick for a very large dataset.

Advantages of Stochastic Gradient Descent

- Learning is much faster than batch gradient descent
- Redundancy is computation is removed as we take one training sample at a time for computation
- Weights can be updated on the fly for the new data samples as we take one training sample at a time for computation

Disadvantages of Stochastic Gradient Descent

- As we frequently update weights, Cost function fluctuates heavily

	Temp.	Visibility	Actual o/p	Predicted o/p
Update w_i	73	10	1	0
Update w_i	50	2	1	1
Update w_i	32	1	0	1
Update w_i	80	5	1	1
Update w_i	23	.1	0	1

Stochastic Gradient descent

Mini Batch Gradient descent

Mini-batch gradient is a variation of stochastic gradient descent where instead of single training example, mini-batch of samples is used.

Mini batch gradient descent is widely used and converges faster and is more stable.

Batch size can vary depending on the dataset.

As we take a batch with different samples,it reduces the noise which is variance of the weight updates and that helps to have a more stable converge faster.

	Temp.	Visibility	Actual o/p	Predicted o/p
	73	10	1	0
Update w_i	50	2	1	1
	32	1	0	1
	80	5	1	1
Update w_i	23	.1	0	1
	32	1	1	0

Mini batch Gradient descent

Advantages of Min Batch Gradient Descent

- Reduces variance of the parameter update and hence lead to stable convergence
- Speeds the learning
- Helpful to estimate the approximate location of the actual minimum

Disadvantages of Mini Batch Gradient Descent

- Loss is computed for each mini batch and hence total loss needs to be accumulated across all mini batches

Model Performance

Model performance in linear regression can be evaluated using various metrics that measure how well the model fits the data and how well it generalizes to new data. Here are some common metrics used in linear regression:

Example: Suppose we have a linear regression model that predicts housing prices based on the size of the house. We have 10 data points with the following actual prices (y) and predicted prices (y_hat):

y = [100, 150, 200, 250, 300, 350, 400, 450, 500, 550]

y_hat = [110, 140, 180, 240, 290, 320, 380, 420, 480, 520]

y_mean = sum(y) / len(y) = 325

1. Sum of Squares Regression (SSR): SSR is calculated by taking the sum of the squared differences between the predicted values and the mean of the dependent variable. It represents the variability in the dependent variable that is explained by the independent variables. Mathematically, it can be expressed as:

SSR = sum((y_hat — y_mean)²) = 28300

2. Sum of Squares Error (SSE): SSE is calculated by taking the sum of the squared differences between the predicted values and the actual values of the dependent variable. Mathematically, it can be expressed as:

SSE = sum((y — y_hat)²) = 26700

3. Mean Squared Error (MSE): This metric measures the average squared difference between the predicted and actual values. It is calculated as the sum of squared residuals divided by the number of data points. A lower MSE indicates better performance. Mathematically, it can be expressed as:

MSE = (1/n) * sum((y — y_hat)²) = 2670

4. Root Mean Squared Error (RMSE): This metric measures the square root of the MSE and is useful for interpreting the errors in the same units as the dependent variable. Mathematically, it can be expressed as:

RMSE = sqrt(MSE) = sqrt(2670) = 51.68

5. Mean Absolute Error (MAE): This metric measures the average absolute difference between the predicted and actual values. It is less sensitive to outliers than MSE, but still provides a measure of the model's accuracy. Mathematically, it can be expressed as:

MAE = (1/n) * sum(abs(y — y_hat)) = 36

6. Sum Of Squares Total (SST): SST is calculated by taking the sum of the squared differences between the actual values of the dependent variable and its mean value. Mathematically, it can be expressed as:

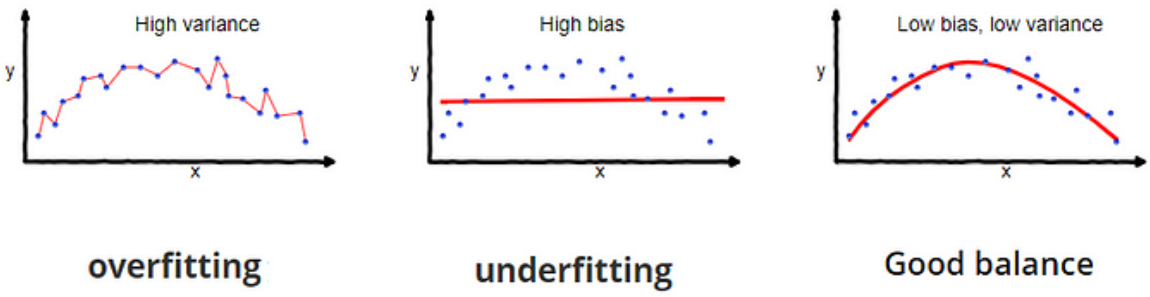
SST = sum((y — y_mean)²) = 55000

6. R-Squared (R^2): This metric measures the proportion of variance in the dependent variable that is explained by the independent variable(s). It ranges from 0 to 1, with a higher value indicating better performance.

R^2 = SSR / SST = 28300 / 55000 = 0.5164

R^2 = 1 — SSE / SST = 1 —26700 / 55000 = 0.5164

Bias and Variance:

Bias: The error due to overly simplistic assumptions in the learning algorithm. It reflects the inability of the model to capture the true patterns in the data, leading to systematic errors in predictions (e.g., underfitting).

Variance: The error due to sensitivity to small fluctuations in the training set. It measures how much the model's predictions change when using different training data — high variance means the model performs well on training data but poorly on unseen/test data (e.g., overfitting).